IRWA Final Project - Part 1

Text Processing and Exploratory Data Analysis

Group members: Andreu Clotas Burgas, u232071 · Elena Andreev Apukhtina u231981 · Carolina Henao Fernandez, u232758
Repository: github.com/elenagotlost/irwa-search-engine
tag/commit: IRWA-2026-part-1

The corpus contains 28,080 fashion products crawled from Flipkart, each with a unique pid and 17 fields.
All the processing code lives in `myapp/core/preprocessing.py` so that Parts 2-4 reuse exactly the same pipeline for documents and queries.
The notebook `project_progress/part_1/part_1.ipynb` runs it, justifies the decisions with data and produces every number and figure in this report.

1. Data preparation

1.1 Text pipeline (title, description, product details, brand)

The same function, `preprocess_text`, is applied to documents and to queries, otherwise query terms and index terms would not match.

The steps are:

- **Unicode:** ASCII folding and lower-casing ("Café" → "cafe").
- **Domain normalisation** (bonus): spelling variants of the same clothing piece are unified before tokenizing (t-shirt/t shirt/tshirt/tee = tshirt, sweat shirt = sweatshirt, hoody = hoodie, jean = jeans), and glued compounds found in the details are split (slimfit = slim fit, fullsleeve = full sleeve), so the query term slim matches them.
- **Punctuation removal**: every non-alphanumeric character becomes a space, not an empty string. Many descriptions glue sentences together ("rich look.Designed for Comfort,Skin friendly"); deleting the punctuation would create non-words such as lookdesigned. Apostrophes are dropped, so men's → mens → men.
- **Tokenization** with NLTK `word_tokenize`.
- **Stop-word removal:** with the NLTK English list, plus single characters and pure numbers ("100" from "100% cotton", "2" from "Pack of 2").

  Numbers are handled as typed fields (Section 1.4). Alphanumeric tokens such as 4xl are kept. We checked that the list does not remove the words the queries depend on (men, women, full, slim, colours).
- **Porter stemming** (cached). For example: printed = print, sleeves = sleev.

Step	Tokens	Vocabulary	Vocabulary reduction
raw whitespace split	1,017,425	13,521	—
normalised + tokenized	1,023,834	6,545	51.6 %
stop words removed	690,067	6,233	53.9 %
stemmed	690,067	4,750	64.9 %

Normalisation is the most effective step: it halves the vocabulary because the raw text has many variants of the same word (Cotton, cotton, cotton.Designed). Stop-word removal barely changes the vocabulary but removes a third of all tokens: stop words are few but very frequent, so removing them makes the postings lists much shorter.

1.2 Output fields

Every processed record keeps the 14 fields that results must return, in their original form: *pid*, *title*, *description*, *brand*, *category*, *sub_category*, *product_details*, *seller*, *out_of_stock*, *selling_price*, *discount*, *actual_price*, *average_rating*, *url*.

product_details is flattened from a list of one-key dictionaries into a single dictionary.

Next to them we store the token fields, the keyword fields and the parsed numeric fields described below. The processed corpus is saved to `data/processed_corpus.pkl` as the input for Part 2.

1.3 category, sub_category, brand, product_details and seller

We measured how distinctive each field is (number of values, normalised entropy, IDF of a value if it were indexed as a term) and where the validation queries actually find their terms.

Field	Distinct values	Top value share	Norm. entropy	IDF of top value
category	4	96.6 %	0.11	0.03
sub_category	22	54.5 %	0.51	0.61
brand	321	3.6 %	0.82	3.39
seller	534	5.3 %	0.79	2.99

Query	Label	title	description	details	brand	sub_category	all text
1: women full sleeve sweatshirt cotton	0	0.46	0.63	0.57	0.00	0.00	1.00
	1	0.80	0.58	0.68	0.00	0.00	1.00
2: men slim jeans blue	0	0.72	0.40	0.48	0.00	0.00	0.80
	1	0.80	0.35	0.65	0.00	0.00	0.90

Share of query terms found in each field, averaged over relevant (1) and non-relevant (0) products of validation_labels.csv.

What the data shows:

- **A single merged field cannot separate relevant from non-relevant products**: for query 1 both groups contain all the query terms somewhere (all text = 1.00). An AND query lets both through, so the ranking must know where a term matched.
- **The title is the most discriminative field** (0.80 vs 0.46 for query 1). Most non-relevant products for query 1 are T-shirts that mention "sweatshirt" or "full sleeve" only outside the title.
- **The title is not enough on its own**: cotton appears in the title of none of the relevant products of query 1, but in 92 % of their descriptions and 69 % of their

details.

- **Gender decides query 2**: most non-relevant results are women's slim blue jeans that only miss men. Gender is in the title and in the Ideal For detail, never in brand or sub_category.
- **category** is almost constant; sub_category values are broad ("Topwear"), never contain query terms and are partly noisy (seller names such as "SUNSHOPPING Bags, Wallets & Belts"); brand and seller are distinctive but only useful if the user asks for them, and a third of the brand names are truncated in the crawl ("Pu" = probably Puma, "Lev" = probably Levi's…).

Option	Pros	Cons
A. Merge all fields into one text field	Simplest index and query processing; any term matches wherever it appears.	Frequent values (clothing, accessories, topwear) flood the postings lists; a match in a seller name counts as much as one in the title; the ranking cannot tell fields apart.
B. Separate indexed fields (fielded index, e.g. BM25F)	Each field can get its own weight (title > details > description); field queries (brand:puma).	Larger index and more complex scoring; weights must be tuned; low-information fields stay useless.
C. Keyword fields for filters / facets	Exact matching for fields with few values or names; cheap filters and facets in the UI.	A value typed in the query ("footwear") is not matched as text; truncated names are hard to search.

Decision. We combine the options according to how distinctive each field is. **title**, **description**, the informative **product_details** values and **brand** are kept as separate token fields (option B) and concatenated into **all_tokens**. The merged field serves the AND filter and the first TF-IDF ranking of Part 2, and the separate fields allow Part 3 to weight the title more. **category**, **sub_category** and **seller** are kept only as normalised keywords (option C) for filters and display; brand is both text and keyword.

The accepted risk is that a query word that only exists as a sub-category ("winter wear") is not matched as text; such words usually appear in titles and details too.

For **product_details** only the values are indexed. Values of non-informative keys are dropped: **codes**, **care instructions**, **packaging**, **country of origin**, **warranty**, **measurements** and the **secondary colour**. Keys are compared without case, because the data spells some of them in several ways ("Pack of" / "Pack Of"). Yes/No attributes become the attribute name ({Hooded: Yes} → hooded) and repeated values (Color and Brand Color) are kept once. The free-text "**Other Details"** is kept unless it is a washing instruction, because it is often a short seller description, with it, women appear in 23% of the details of the relevant query-1 products instead of 0.

1.4 Numeric fields: out_of_stock, selling_price, discount, actual_price, average_rating

These fields arrive as strings ("2,999", "69% off", "") and are not indexed as text: as terms, "499" would match unrelated products, the IDF of a number means nothing, and needs such as "shirts under 500" are ranges that exact term matching cannot express.

Each field is parsed into a typed column kept next to the index:

Field	Stored as / missing values	Planned use (Parts 3–4)
selling_price	float; 2 missing	range filter; price factor in the custom score, relative to the sub-category
actual_price	float; 777 empty → set to the selling price (flag price_imputed)	recompute the discount; display
discount	float %; recomputed from the prices, 0 when there is no list price	filter ("at least 50 % off"); small bonus in the custom score
average_rating	float in [1, 5]; 2,261 kept as NaN + flag has_rating	quality factor, smoothed towards the mean (3.63) since there is no review count
out_of_stock	bool	demote (not remove) unavailable products; optional filter

Missing ratings are not set to 0 (that would mean "very bad") nor to the mean (that would invent information). The ranker decides explicitly how to treat unrated products.

2. Exploratory data analysis

2.1 Missing data.

No field is null, but many are empty strings: the description is empty for 39.7 % of the products, the rating for 8.1 %, the brand for 7.2 % and the seller for 5.9 %. Title, category, sub-category, details and URL are always present, so title and details must carry enough information on their own.

2.2 Text length.

Titles are short and follow a template: pattern + gender + type + colour ("Printed Men Round Neck Black T-Shirt") with a median of 6 words.

Descriptions have a median of 10 words (max 623), counting empty descriptions and have few medium-length sentences (median 2 sentences of 14 words).

The details give a steady approx. 13 terms. Because lengths differ so much between fields, length normalisation and per-field weighting both matter.

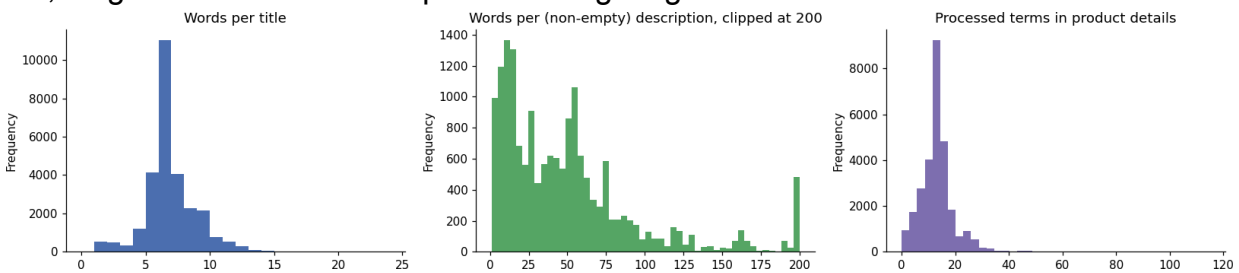


Figure 1. Words per title, words per non-empty description (clipped at 200) and processed terms in the details.

2.3 Vocabulary.

After processing, all fields together have **5,710 terms**; 1,342 (23 %) appear in a single product.

The most frequent title terms are the template dimensions (women, men, tshirt, neck, solid, round, print); descriptions are dominated by marketing words (wear, comfort, look, casual, quality) that are frequent everywhere and get a low IDF.



Figure 2. Word clouds of titles and descriptions (unstemmed tokens, no stop words)

The word clouds show that titles describe who and what while descriptions describe how it feels (cotton, fit, comfortable): the two vocabularies are complementary, which supports keeping them as separate fields.

2.4 Zipf and Heaps.

The rank/frequency curve is close to a straight line with slope −1.14 (Zipf predicts about −1). The vocabulary grows very slowly: Heaps' law fits with **β = 0.29** (K = 106.8), well below the 0.4 - 0.6 typical of news collections, and the first 10% of the products already contain 53% of the final vocabulary. The corpus is very repetitive, so the inverted index will have few terms but very long postings lists for words such as tshirt, men or cotton.

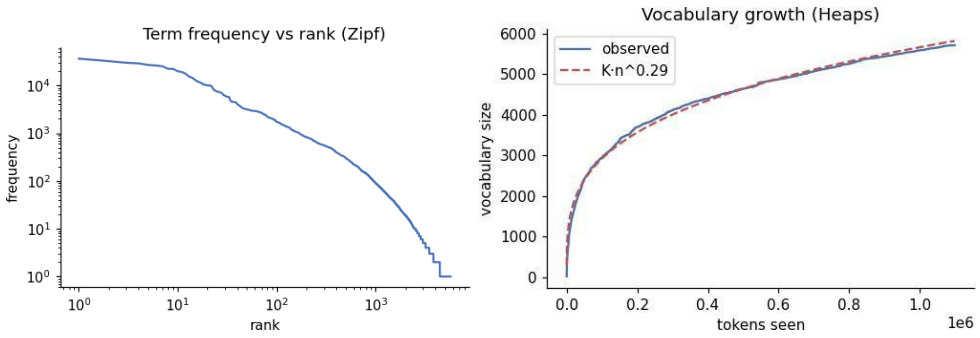


Figure 3. Zipf (rank vs frequency, log-log) and Heaps (vocabulary growth) curves of the corpus.

2.5 Categories, brands and sellers.

Sub-categories have a long tail (Topwear alone has 15,310 products; about ten have fewer than 30). Brands and sellers are spread evenly: the largest brand (ECKO Unl) is only 3.6 % of the corpus, and the largest seller (RetailNet) sells many brands, so seller and brand are different dimensions. Brand names are often truncated (34.7 % of

products have a brand of 4 characters or fewer).

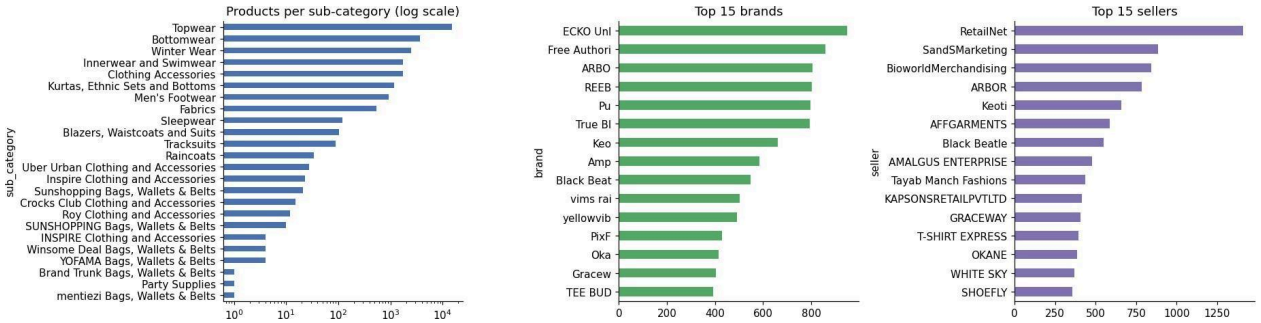


Figure 4. Products per sub-category (log scale), top 15 brands and top 15 sellers.

2.6 Price, discount, rating and stock.

Prices are skewed (median Rs. 545, 75% below Rs. 820, max Rs. 7,999). Discounts are large (median 52%) and ratings are concentrated between 3.2 and 4.1 (mean 3.63). Only 5.9% of the products are out of stock, but the rate depends on the sub-category: clothing accessories 16.9%, tracksuits 14.4%, winter wear 9.5% (probably seasonal). Median prices depend strongly on the sub-category (Rs. 1,799 for blazers and suits, Rs. 499 for topwear), while the median rating is 3.7 - 3.8 almost everywhere, so a price is only "high" relative to its sub-category.

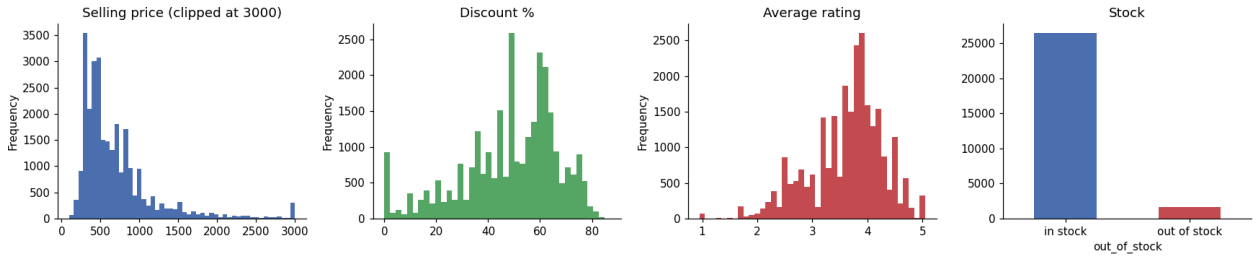


Figure 5. Selling price (clipped at 3,000), discount, average rating and stock.

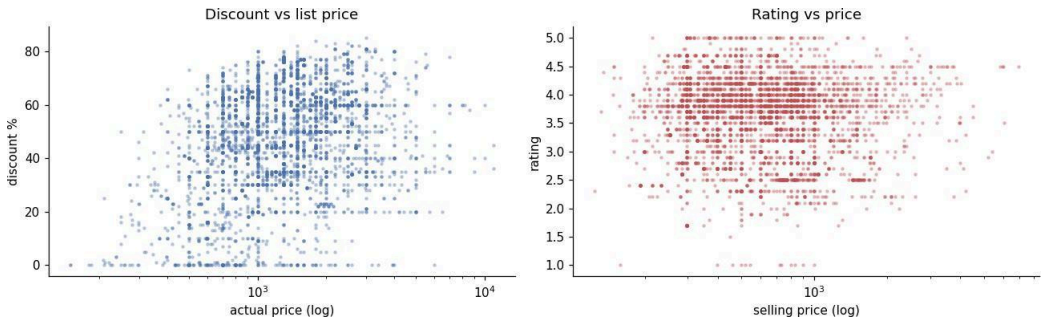


Figure 6. Discount vs list price and rating vs selling price (5,000-product sample, log x-axis).

2.7 Rankings and correlations.

The top-rated products are cheap T-shirts with a 5.0 rating, many from one brand (PixF). The dataset has no review count, so a 5.0 may come from a single review; averaging over brands with at least 50 rated products is more stable (SORA 4.61, PixF 4.46, Cots 4.44).

The most expensive products are tracksuits, blazers, jackets and branded jeans (Rs. 6,500 - 8,000); the largest discounts (84 - 87 %) are on T-shirts costing Rs. 128 - 280. Spearman correlations show that selling and list price are related (0.75) and that discount falls with price (−0.37), which suggests inflated list prices on cheap items.

Rating is unrelated to price and discount (|ρ| ≤ 0.09), so each can be a separate factor in a custom score.

2.8 Duplicates.

87.1% of the products share their title with another product and 56.2% share title, description and brand: the same garment in other colours or sizes has its own pid ("Slides" alone appears 544 times). We keep all of them, since pids are needed for evaluation, and note that the ranking may need diversification so the top results are not near-identical.

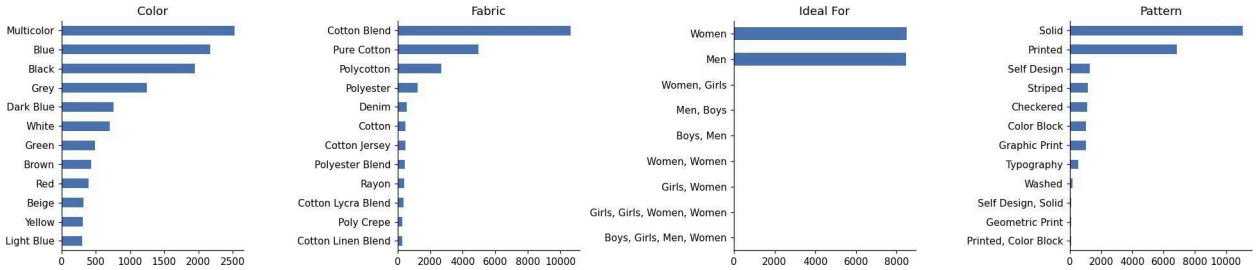


Figure 7. Most frequent values of four product_details attribute

2.9 Entity recognition.

spaCy's general English model (en_core_web_sm, trained on news) performs poorly on 2,000 sampled products. It labels titles as organisations ("Printed Women Round Neck White"), fabrics and colours as places ("Cotton", "Maroon") and garments as people ("Track Pant", "Boxer"), because product text has no sentence structure and every word is capitalised. In this domain the reliable entities are the structured attributes in product_details: colour, fabric (Cotton Blend, Pure Cotton), target group (Women and Men in almost equal numbers) and pattern. These are exactly the facets of the queries, which is why their values are indexed as text.

Use of AI tools

We used Claude (Anthropic) during this part of the project. Claude generated the initial version of the pre-processing module (`myapp/core/preprocessing.py`) and the structure of the EDA notebook, helped us debug specific code and environment problems, and suggested ways to analyse the data and present the results. We reviewed, ran and modified all the code. Our changes include the handling of product_details (case-insensitive keys, keeping “Other Details”) and the Zipf and Heaps analysis. We also checked every number in this report against our own notebook outputs. The interpretation of the results and the written analysis are our own, and all group members understand and can explain the submitted code.